改进 JPS 算法融合 DWA 的多机器人路径规划*

任祥瑞，王正刚†，汤俊杨
（安徽工程大学 电气工程学院，安徽 芜湖 241000）

摘　要：针对传统的跳点搜索（jump point search，JPS）算法在移动机器人路径规划时，存在路径拐点以及中间跳点过多，路径规划时间较长等问题，提出了改进的跳点搜索算法 I-JPS。I-JPS 算法通过改进代价函数、引入叉积公式，来剔除冗余节点、增加机器人与障碍物之间的安全距离。同时引入了动态窗口法（dynamic window approach，DWA）作局部路径规划，用于机器人临时避障和路径平滑化，并通过改进 DWA 提高多机器人之间的避障优先级。最后引入了多机器人协同路径规划，多机器人可以共同合作并完成复杂的任务，机器人之间还可以共享信息、协调行动，并通过分工合作来解决问题，提高任务的完成效率。最后，实验仿真结果表明改进后的算法相较于改进前的，在各方面都得到了极大的提升。

关键词：跳点搜索；路径规划；动态窗口法；代价函数；多机器人
中图分类号：TP242.6　　**文献标志码：**A　　**文章编号：**1001-3695(2024)11-007-3251-07
doi:10.19734/j.issn.1001-3695.2024.03.0099

Improved JPS algorithm fused with DWA for multi robot path planning

Ren Xiangrui, Wang Zhenggang†, Tang Junyang
(*College of Electrical Engineering*, *Anhui Polytechnic University*, *Wuhu Anhui* 241000, *China*)

Abstract: For the traditional JPS algorithm in mobile robot path planning, there are path inflection points as well as too many intermediate jump points, and the path planning time is long. This paper proposed an improved jump point search (JPS) algorithm. The I-JPS algorithm eliminated redundant nodes and increased the safe distance between the robot and obstacles by improving the cost function and introducing the fork product formula. It also introduced DWA for local path planning for temporary obstacle avoidance and path smoothing by robots, and increased the priority of obstacle avoidance between multiple robots by improving DWA. Finally, it introduced multi-robot collaborative path planning, where multiple robots could work together and complete complex tasks, and the robots could also share information, coordinate their actions, and solve problems through division of labour to improve the efficiency of task completion. Finally, the experimental simulation results show that the improved algorithm is greatly improved in all aspects compared to the pre-improved one.

Key words: jump point search; path planning; dynamic window method; heuristic function; multiple robots

0　引言

在现代自动化领域，随着技术的不断发展，机器人系统广泛应用于物流、制造、服务等多个领域，路径规划是机器人和自主系统中至关重要的一环，特别是在多机器人系统中。多机器人系统的路径规划问题涉及到如何高效地协调和规划多个机器人的运动轨迹，以实现任务的顺利完成[1]。机器人路径规划一直是机器人领域至关重要的组成部分，具有理论和应用价值。它帮助机器人解决在简单或者复杂环境下需要应对的困难[2]。路径规划方法可以分为全局路径规划和局部路径规划。全局路径规划包括传统的蚁群算法[3]、A* 算法[4]、粒子群算法[5]、Dijkstra 算法[6]、遗传算法[7]等。局部路径规划包括动态窗口法[8]、人工势场法[9]等。面对不同的复杂环境，选择一个合适的算法对于路径规划来说尤为重要。

朱敏等人[10]利用跳点搜索算法不均匀分配初始信息素，降低蚁群前期盲目搜索的概率；然后，引入切比雪夫距离加权因子和转弯代价改进启发函数，提高算法的收敛速度、全局路径寻优能力和搜索路径的平滑程度；最后，提出一种新的信息素更新策略，引入自适应奖惩因子，自适应调整迭代前、后期的信息素奖惩因子，保证了算法全局最优收敛。周滔等人[11]选择了 A* 作为路径规划算法。建立了具有非完整约束特征的移动机器 人运动学模型和跟踪误差模型，采用改进的自适应轨迹跟踪控制对 A* 运动轨迹进行有效跟踪。刘荣华等人[12]提出改进双向动态 JPS 算法。改进算法动态定义正、反扩展方向上的目标点，动态定义启发函数，并利用动态约束椭圆对算法的扩展区域加以限制，以区分椭圆内、外区域的扩展优先级。苗红霞等人[13]提出了并行-交替式双向跳点搜索算法。采用改进了预计代价函数的并行式双向跳点搜索算法，然后，采用交替式双向跳点搜索算法，规划中心热点区域内部的路径。林信川[14]提出一种融合跳点搜索和双向并行蚁群搜索的改进算法。首先使用改进的跳点搜索算法生成双向搜索的初始次优路径，为双向蚁群搜索提供初始搜索方向参考。其次在双向并行蚁群搜索过程采用改进的转移概率启发函数，该函数在确定下一个转移节点时考虑了避免 AGV 与障碍物碰撞的因素，同时通过设计信息素共享机制并结合改进的信息素增量及浓度两种融合模型。

针对传统跳点搜索算法在路径规划上存在时间过长、搜索效率低等问题。本文提出了改进的跳点搜索算法，该算法通过改进跳点的筛选规则，将障碍物膨化处理，提出了新的叉积拟合公式对路径进行剪枝优化，得到一条性能最优的路径。为考虑

收稿日期：2024-03-11；**修回日期：**2024-05-31　**基金项目：**安徽省高校科学研究重点项目（2022AH050977）
作者简介：任祥瑞（2000—），男，安徽马鞍山人，硕士研究生，主要研究方向为移动机器人路径规划；王正刚（1982—），男（通信作者），安徽滁州人，副教授，硕导，硕士，主要研究方向为信号与信息处理，路径规划等（wangzhenggang@ahpu.edu.cn）；汤俊杨（1998—），男，安徽淮南人，硕士研究生，主要研究方向为移动机器人路径规划控制.

中国知网　https://www.cnki.net

机器人寻路的安全性,本文又引入了障碍率,并将原本的代价函数做了全新的改进,将移动机器人的寻优路径与栅格地图中的随机障碍物保持了一定的安全距离。最后考虑到路径的平滑性,本文又引入了 DWA 局部路径规划[15],使移动机器人规划出的路径更加平滑。

1 传统 JPS 算法及其改进

1.1 环境地图构建

在移动机器人路径规划的研究中,环境建模是一个重要的步骤,它涉及对机器人所在环境的描述和表示。环境建模的目标是为路径规划算法提供一个准确的、可操作的环境表示,以便机器人能够根据这个模型作出决策。

考虑环境建模的简单且有易于实现,本文选用栅格法作为环境建模。如图 1 所示,栅格法使用网格化的地图来表示环境,将整个区域划分为一系列的正方形或矩形网格,每个网格单元表示环境中的一个离散区域,可以是空的、障碍物或其他特定属性。格栅地图中白色方块表示机器人可以自由通过的区域,黑色方块表示障碍物。同时,栅格地图中的障碍物都是进行过膨化处理的,只要被障碍物涉及的区域,无论大小,均用黑色方块表示。栅格法的每一个方块都有它独一无二对应的坐标,具体公式为

$$\begin{cases} x_i = a(\mathrm{mod}(i, N_x) - 0.5) \\ y_i = a(N_y + 0.5 - \mathrm{ceil}(\frac{i}{N_y})) \end{cases} \tag{1}$$

其中:(x_i, y_i)表示第 i 个栅格的位置坐标,x_i 是该栅格在水平方向上的坐标,y_i 是该栅格在垂直方向上的坐标。mod( )表示取余运算,ceil( )表示取整运算,这些运算通常用于将实际坐标映射到栅格地图中的栅格索引,N_x 是栅格地图的行数,N_y 是栅格地图的列数,a 表示每个栅格的边长。

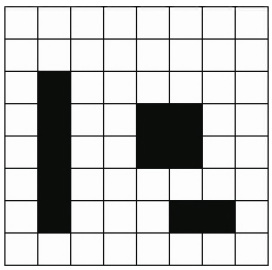


图 1 栅格地图

Fig. 1 Grid map

1.2 传统的跳点搜索算法

跳点搜索算法是一种用于路径规划的高效算法,它在 A* 算法的基础上进行了优化,适用于网格化地图的路径搜索。跳点搜索算法的核心思想是通过预处理和跳跃的方式,减少搜索空间,从而提高搜索效率。搜索过程中,在地图的每个方向上对每个节点进行预处理,标记出可以进行跳跃的节点,JPS 会跳过一些明显不可能包含最优路径的节点,探索真正可能包含最优路径的节点,从而减少搜索开销[16]。

跳点搜索算法需要定义节点 P 的自然邻节点与强制邻节点,如图 2 所示,灰色栅格即表示没有被障碍占据的自由空间,白色栅格表示节点 P 的自然邻节点,蓝色栅格表示节点 P 的强制邻节点,如图 2(c)(d)所示。

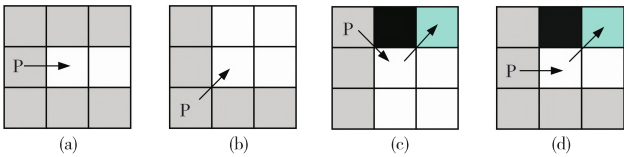


图 2 传统邻节点裁剪规则

Fig. 2 Traditional neighbor node pruning rules

关于了解跳点搜索算法,需要了解它的定义,强迫邻居和自然节点。强迫邻居指的是从一个节点出发,在向邻居节点探索的过程中,由于障碍物的存在,某个邻居节点成为“强迫”的,它是被迫被探索的,而不是自然地成为探索路径的一部分。在 JPS 算法中,当算法发现一个自然节点后,会探索该节点的邻居,如果某个邻居节点被发现是一个强迫邻居,算法会将其添加到路径中。这种策略帮助算法在规划路径时更好地适应和绕过障碍物,提高了路径的质量和效率,而在无障碍物环境中,查询的节点称为自然节点。

1.3 改进的跳点搜索算法

1.3.1 改进启发函数

跳点搜索算法结合了 A* 与 Dijkstra 算法的广度优先搜索和贪心算法的启发式搜索,通过一个估价函数来评估搜索过程中每个节点的优先级,有效地在搜索空间中选择下一个要扩展的节点,从而达到减少搜索范围的目的。其中总的评价函数为

$$f(n) = g(n) + h(n) \tag{2}$$

其中:n 表示路径的当前节点,$f(n)$表示总的代价函数,是从起始点到目标节点的总代价,它的值越小则越优。$g(n)$表示实际代价函数,它表示从起始点到节点 n 的实际代价,是固定不变的。$h(n)$是估计函数,表示从当前节点 n 到目标节点的预计代价成本,它往往是代价函数最重要的一部分。

为提高传统的 JPS 算法在路径规划中的效率,将改进其代价函数,代价函数在路径规划中是至关重要的,因为它直接影响算法选择扩展的节点,可以结合实际代价和启发式代价,以更准确地确定节点的优先级。改进后的代价函数为

$$f(n) = g(n) + \left(1 + \frac{\alpha}{\beta}\right) e^{Q + h(n)} \tag{3}$$

其中:α 为机器人当前位置到目标点的距离;β 为起始点到目标点的距离。Q 表示障碍覆盖率,为

$$Q = \frac{p}{P} \times 100\% \tag{4}$$

其中:P 表示整个栅格地图中所有的方块,包括白色机器人可以自由通行方块和黑色障碍物方块的个数;p 则表示黑色障碍物的方块的个数。Q 即被障碍物所占据的方格数量占总方格数量的比例,用于量化地图中障碍物的密集程度,障碍率越高,表示地图中的障碍物越密集,机器人在规划路径时更加复杂,需要更谨慎地避免这些区域。在栅格地图的导航和路径规划中,障碍率有助于机器人系统更好地理解环境,提高决策的准确性和效率。

如图 3 所示,引入新的代价函数后,改进了传统跳点搜索算法中存在的多处危险节点等情况,使机器人与障碍物之间保持了合适的安全距离,让规划出的路径安全性得到了极大的提升。通过去除危险节点,可以提高路径规划系统的可靠性,确保机器人在移动过程中不会遇到潜在的危险或障碍。同时跳点的辐射范围变大,让算法搜索目标的能力增强,迭代次数变少,搜索速度增加。

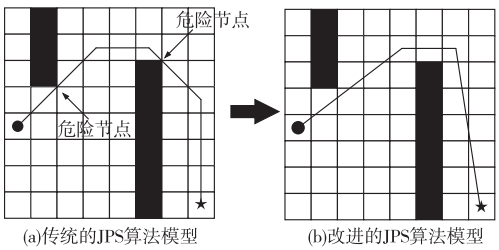


图 3 模型改进

Fig. 3 Model improvements

中国知网 https://www.cnki.net

1.3.2 路径节点优化

对于改进前的 JPS 算法,存储了大量无用的跳点,导致计算量大、冗余节点多、路线过于靠近障碍物等一系列问题,本文提出了一种基于线性插值的剪枝算法,提高了运行效率,让规划出来的路径得到最优的效果。改进的具体步骤如下所示。

a)对 JPS 算法规划出来的路径的每一个点进行遍历,利用本文的改进算法,遍历路径中的每一个节点,对任意两个节点进行连线,即为剪枝后的路径。如图 4 所示,利用剪枝法剪枝前的路径是 A→B→C→D,对 AC 两点进行剪枝,剪枝后的路径为 A→C→D,再次剪枝后为 A→D。

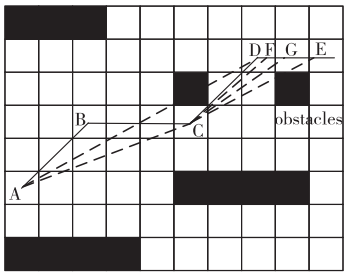


图 4　剪枝的过程

Fig. 4　Pruning process

b)剪枝后的路径并非为最终路径,还需要检查当前节点 (X_N, Y_N) 与后面第二个节点 (X_{N+2}, Y_{N+2}) 以及后面节点之间的路径是否与障碍物发生碰撞。

c)将当前节点 (X_N, Y_N) 与后面第二个点 (X_{N+2}, Y_{N+2}) 利用改进算法后的路径与所有障碍物进行碰撞检测,再利用向量叉积的符号判断两线段是否相交。叉积公式如下:

$$\partial = (A_X - C_X)(D_X - C_X) * (D_X - C_X)(A_X - C_X) \tag{5}$$

假设线段 AC 与 CD 相交,那么其叉积结果 ∂ 的符号将决定它们是否相交。如果叉积为正值,则表示相交,否则不相交。如图 5 所示,可以是 AC,也可以是 AD。本文算法将剪枝后的路线与所有障碍物的边进行叉积运算,作为碰撞检测。

d)根据叉积结果 ∂ 判断,如果叉积为负,则说明剪枝成功,未发生碰撞,如图 4 中的 AC 路径,则表示剪枝成功了。再保留优化后的路线,删除原始的多余路径。如图 4 中 AD 路径所示,发生了碰撞,则优化失败,删除路径上优化的路径,并继续检查优化后的路径,直到整个路径被优化完成。

e)改进算法在剪枝过程中并不局限于每个拐点之间的剪枝。它可以在一条线段中截取需要的点作为剪枝点。DE 是一条线段,拐点 C 可以在线段 DE 中的任意一点来连线,连线 CE,线段 CE 与障碍物发生碰撞,继续检测到线段 DE 中的 G 点连接 CG,线段 CG 并没有与障碍物发生碰撞。同时该算法也可以检测到除 DE 上 G 点之外的,例如 F 点,但 F 点不优于 G 点,所以该算法选取 G 点作为最优剪枝点。

经过这个优化步骤,路径中的多余中间节点会被删除,以避开障碍物,从而得到一条更加优化的路径。在计算总路径的长度,本文依然使用欧氏距离,并对这些距离进行累加。

$$\begin{aligned} S_1 &= \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2} \\ S_2 &= \sqrt{(X_3 - X_2)^2 + (Y_3 - Y_2)^2} \\ &\vdots \\ S_n &= \sqrt{(X_{n+1} - X_n)^2 + (Y_{n+1} - Y_n)^2} \end{aligned} \tag{6}$$

$$S_{\text{line}} = S_1 + S_2 + \cdots + S_n \tag{7}$$

其中:S_1 为第一个点 (X_1, Y_1) 到第二个点 (X_2, Y_2) 之间的距离,S_2 为第二个点 (X_2, Y_2) 到第三个点 (X_3, Y_3) 之间的距离,直到 S_n,S_{line} 将这些距离进行累加处理,为路线的总长。算法的具体流程如下:

a)栅格地图初始化,将起点与终点确定,选择合适的障碍物膨胀尺寸,并确定好障碍物的覆盖率方便后面计算。

b)初始化空的 openlist 数组,用于存储可扩展的节点。初始化 flag = 1,表示邻居节点未在障碍物列表中出现。将起点加入 openlist 中,根据改进后的最优 $f(n)$,寻找 openlist 中 f 值最小的点,为 current。

c)判断当前点 current 是否为最终目标点,如果是,则寻路结束,最终路径生成,无须进行以下步骤。如果不是,继续进行。

d)如果 flag 仍然为 1,表示邻居节点有效且不在障碍物列表中,计算邻居节点的启发式估计值 $h(n)$ 和综合代价 $f(n)$。在 openlist 中删除 current,closelist 中加入 current 当前点。

e)继续探索下一个 current 点,在 openlist 中选择具有最小 $f(n)$ 值的节点,作为下一个要扩展的节点。对于 current 在八个方向的邻居点 neighbor 进行判断,如果 neighbor 不可通过或者已经存在于 closelist 中,跳过。如果不在 openlist 中,则加入,如果存在,通过 G 值判定 neighbor 是否为父节点 current,并更新 G 值。重复步骤 c)d),直到当前点为终点。

f)遍历 openlist 中所有跳点,判断跳点间是否存在障碍物,不存在则利用多边剪枝算法删除多余的跳点。不断更新迭代后形成最终路径,规划完成。

2　动态窗口法及改进

2.1　机器人运动模型分析

移动机器人在进行路径规划时会遇到很多难以预料的复杂情况,静态路径规划已经不能满足移动机器人对动态障碍物的躲避,而动态窗口法可以解决,对移动的障碍物进行有效实时的躲避。动态窗口法通过在速度空间中搜索合适的速度和方向组合,考虑机器人的动力学约束,以找到在给定时间范围内的最佳运动策略。它具有简单、实时性强等优点。同时,动态窗口法使用一个“窗口”来限制速度空间的搜索范围。这个窗口基于机器人当前状态和动力学特性,包括最大线速度、最大角速度和加速度限制等,窗口的设计是为了保持搜索的效率和合理性[17]。

动态窗口法是基于运动学模型的一种经典的局部路径规划算法,通过对机器人的线速度和角速度进行采样,并在采样周期内对移动机器人的运动轨迹进行模拟。如果移动机器人运动轨迹是圆或者是直线,那么速度空间中的线速度和角速度变化可以反映机器人的运动状态,机器人的运动模型如下:

$$\begin{cases} x_t = x_{t-1} + \nu_x \Delta t \cos\theta_t - \nu_y \Delta t \sin\theta_t \\ y_t = y_{t-1} + \nu_x \Delta t \sin\theta_t + \nu_y \Delta t \cos\theta_t \\ \theta_t = \theta_{t-1} + \omega_t \Delta t \end{cases} \tag{8}$$

其中:x_t, y_t, θ_t 分别代表了移动机器人在世界坐标系下的位置姿态。

2.2　速度采样

速度采样是 DWA 算法中的一个关键步骤,它涉及在机器人当前状态下对速度空间进行离散采样,生成一系列可能的速度和方向组合,以在这些组合中寻找最优的运动轨迹。根据自动引导车的自身要求和其他环境的要求,对采样的条件进行限制,并限制在一个合理的范围之类,需要满足以下条件:

a)移动机器人最大最小速度的约束。

移动机器人的线速度采样和最佳角速度采样应控制在最佳区间,约束如下:

$$v_m = \{(\nu, \omega) \mid \nu \in [v_{\min}, v_{\max}], \omega \in [\omega_{\min}, \omega_{\max}]\} \tag{9}$$

其中:$v_{\min}$、$v_{\max}$ 为移动机器人最小、最大线速度;$\omega_{\min}$、$\omega_{\max}$ 为移动机器人最小、最大角速度。

b)移动机器人加减速的约束。

中国知网　https://www.cnki.net

机器人的速度采样应考虑电机性能,机器人的加速度和减速度都在电机性能允许的范围内。具体而言,这涉及到的关键约束如下:

$$V_d = \begin{cases} (v,w) \mid v \in [v_c - \dot{v}_b \Delta t, v_c + \dot{v}_a \Delta t] \\ w \in [w_c - \dot{w}_b \Delta t, w_c + \dot{w}_a \Delta t] \end{cases} \tag{10}$$

其中:v_c、ω_c 表示机器人当前的线速度和角速度;$\dot{v}_c$、$\dot{\omega}_c$ 表示机器人最大线加速度和最大角加速度;$\dot{v}_b$、$\dot{v}_b$ 表示机器人最大线减速度和最大角减速度。

c)移动机器人安全距离的约束。

为确保移动机器人在接近障碍物时的安全性,必须在机器人到达障碍物之前采取最大减速度,使其速度迅速降为零。这样做可以确保机器人在靠近障碍物时能够迅速停止运动,从而避免与障碍物发生碰撞。其公式为

$$V_a = \begin{cases} (v,w) \mid v \leqslant \sqrt{2 \cdot \mathrm{dist}(v,w) \cdot \dot{v}_b} \\ w \leqslant \sqrt{2 \cdot \mathrm{dist}(v,w) \cdot \dot{w}_b} \end{cases} \tag{11}$$

其中:$\mathrm{dist}(v,\omega)$ 为 (v,ω) 对应轨迹障碍物的最近距离。综合以上,动态窗口法的最终速度为

$$v_r = v_m \cap v_d \cap v_a \tag{12}$$

2.3　评价函数及优化

动态窗口法的评价函数通常被设计为一个代价函数,目的是根据机器人当前状态和运动窗口内的线速度、角速度组合来评估每个可能的运动指令。代价函数的设计影响了最终路径规划的质量,通常会包括与目标点的距离、与障碍物的距离、线速度角速度、动态窗口大小等多个方面的考虑。其具体的评价函数公式为

$$G(v,w) = \sigma(a_1 \cdot \mathrm{heading}(v,w) + a_2 \cdot \mathrm{dist}(v,w) + a_3 \cdot \mathrm{velocity}(v,w)) \tag{13}$$

其中:$\mathrm{heading}(v,\omega)$ 表示方位角评价函数,为当前速度下轨迹切线与目标点的夹角;$\mathrm{dist}(v,\omega)$ 表示距离评价函数,为当前速度下轨迹与静态障碍物的最近距离;$\mathrm{velocity}(v,\omega)$ 表示速度评价函数,为当前规矩速度的大小。σ 为平滑系数,a_1、a_2、a_3 为各项的加权系数。

传统的 DWA 评价函数在移动机器人局部路径规划时可能存在避障过于激进,导致规划路径过分迂回,影响了路径的快速性。容易陷入局部最优解,而难以全局搜索到更优的路径等情况。因此,通过增加 $\mathrm{dist2}(v,\omega)$ 和 $\mathrm{dist3}(v,\omega)$ 两个函数,$\mathrm{dist2}(v,\omega)$ 表示评价模拟轨迹终点到动态障碍物的最近距离,通过增加对动态障碍物的识别能力,提高机器人躲避障碍物的灵敏度。$\mathrm{dist3}(v,\omega)$ 则表示评价模拟轨迹终点到未知障碍物的最近距离,为了提高避障的灵敏度。具体改进的评价函数如下:

$$G(v,w) = \sigma(b_1 \cdot \mathrm{heading}(v,w) + a_3 \cdot \mathrm{velocity}(v,w) + a_2 \cdot \mathrm{dist1}(v,w) + a_4 \cdot \mathrm{dist2}(v,w) + a_5 \cdot \mathrm{dist3}(v,w)) \tag{14}$$

$$b_1 = a_1 \cdot [1 + \mathrm{e}^{(\frac{R}{\mathrm{dist}k})}] \tag{15}$$

其中:a_4、a_5 为新增项的加权系数;R 为移动机器人转弯时的半径;k 为变量,其值为 1 或 2 或 3,表示 dist1、dist2、dist3 机器人遇到静态障碍物、动态障碍物以及未知障碍物的三种不同情况。引入新的系数 b_1,增加了停车等待状态的 DWA,当多台机器人同时运行并产生路径交叉时,较长路径的机器人将主动停车等待,以优化行进序列,确保较短路径的机器人能够更快地完成任务,以提高灵活性。该算法能够让机器人在动态环境下避免规划不合理的路径,且在遇到动态障碍物时能及时作出正确应对,并优化了动态窗口法。

同时,改进前移动机器人根据静态路线的拐点作为导航点,改进后的 DWA,导航点自适应地均匀插入静态路线的每一个存在的点,从前往后,依次作为移动机器人的目标导航点。极大地减少了机器人运动时路径规划的误差,局部动态路径规划的准确率和精度得到了很大的提升。

3　多机器人及融合算法

3.1　多机器人

对于多机器人,相比于单机器人,多机器人系统可以并行执行任务,从而提高整体效率。每个机器人可以专注于执行特定的子任务,加快任务完成速度。多机器人系统具有冗余性,即使其中的某个机器人出现故障或受到干扰,其他机器人仍然可以继续执行任务,提高系统的鲁棒性和稳健性。多机器人系统还可以同时覆盖更大的区域,从而适用于大规模的任务,如搜索与救援、勘探、环境监测等。

多机器人进行路径规划时,需要在考虑彼此之间的相互作用和路径冲突,防止它们在相同的位置或路径上交叉。同时,多机器人在实时性要求高的情况下,路径规划需要考虑到动态环境中的变化,及时调整机器人的路径以适应新的情况。

如图 5 所示,为多机器人路径规划时可能会出现的四种交叉情况。引入优先级法,主要思想是当两个机器人出现路径交叉时,优先级低的机器人给优先级高的让路,优先级高的机器人先行,并把优先级低的机器人当成障碍物处理。具体来说,系统为每个机器人或任务分配一个优先级,高优先级的机器人或任务具有更高的执行权,这可以确保在紧急情况下,高优先级的任务或机器人能够更快地完成,从而维护系统的整体效率。对于本文的路径规划算法,设置离目标点越远的机器人优先级越低,确保路径较短的率先完成任务,提高工作效率。

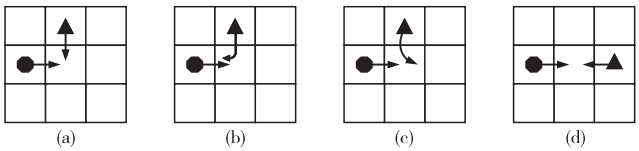


图 5　多机器人路径冲突类型

Fig. 5　Multi-robot path conflict types

除了引入了避障优先级思想,还引入了多机器人分布式任务分配,如图 6 所示,机器人的任务分配一般有集中式和分布式任务分配,集中式任务分配有一个机器人担任主机器人,它与下面的子机器人进行信息交互,但子机器人之间没有任何联系,分布式任务分配不需要主机器人,由子机器人相互之间进行信息交涉,两两之间都可以进行交流,对于本文的路径规划更为合适。

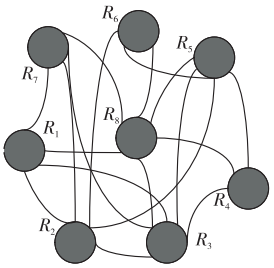


图 6　分布式任务分配

Fig. 6　Distributed task allocation

3.2　融合 JPS 算法与 DWA 及多机器人

将改进后的 JPS 算法融合多机器人再融合改进后的 DWA,目的是针对 JPS 算法和 DWA 的优点和缺点进行相结合,优缺互补。改进的 JPS 算法得到了静态已知障碍物环境下的最优解,但对于动态障碍物和未知障碍物,无法处理,DWA 有良好的局部路径规划能力,但容易陷入局部最优解,所以将两者相结合。同时,多机器人系统可以同时执行多个任务,提高任务的并行性和整体完成效率。

中国知网　https://www.cnki.net

改进后的 JPS 算法结合了全局规划路径和 DWA 算法,通过提取全局规划路径上的关键点作为 DWA 算法的中间指引点,实现在动态环境中为 DWA 算法提供方向,避免陷入局部最优的状况。该优化后的 DWA 算法还能够有效区分动态障碍物和静态障碍物,从而消除一些已知障碍物对路径的干扰,进一步提高算法运算速度。整体而言,这种融合算法结合了两种算法的优点,既具备避障功能,又能够规划出最短路径,从而在复杂环境中表现得更为灵活和高效。具体流程如图 7 所示。

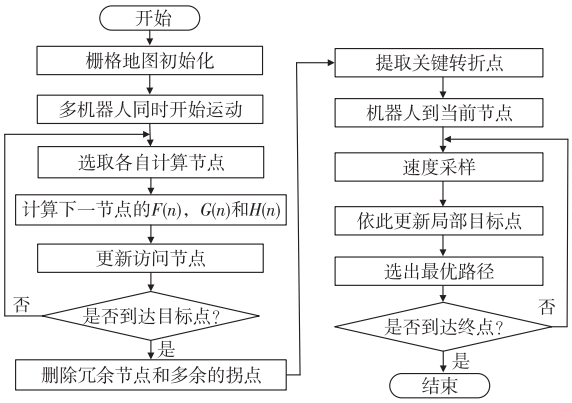


图 7　融合算法的流程

Fig. 7　Flowchart of fusion algorithm

4　实验仿真

4.1　单机器人路径规划仿真实验

为了验证本文提出的 I-JPS 算法的有效性和性能,进行了一系列的实验仿真。仿真的设备配置为 Windows 11 专业版,处理器为 Intel Core i5-11400H,仿真运行平台为 MATLAB2021a。

为了验证实验的可行性,在 MATLAB 平台分别搭载了 20×20 和 30×30 的栅格地图,分为简单环境和复杂环境。简单环境为单机器人路径规划,复杂环境为多机器人路径规划,多机器人实验设置了三个机器人协同规划。

针对文献[16]在路径规划时,路径长度过长以及文献[17]危险节点过多问题,并验证本文 I-JPS 算法的高效性,将原始的两种算法,文献[16,17]两种算法和本文算法在相同的实验条件下进行一系列的比对。首先进行 20×20 地图的简单环境单机器人静态障碍物实验仿真。如图 8 所示,分别将原始的 A* 算法、原始的跳点搜索算法、文献[16,17]的改进算法和本文的改进算法在 20×20 的栅格地图环境下进行一系列对比,图中三角形表示移动机器人的起始点,圆形表示目标点,黑色方块表示静止障碍物。其实验结果如表 1 和图 9 所示。实验进行了 30 次,最终的运行时间、路径长度、拐点数和搜索节点均按取 30 次平均数所得。

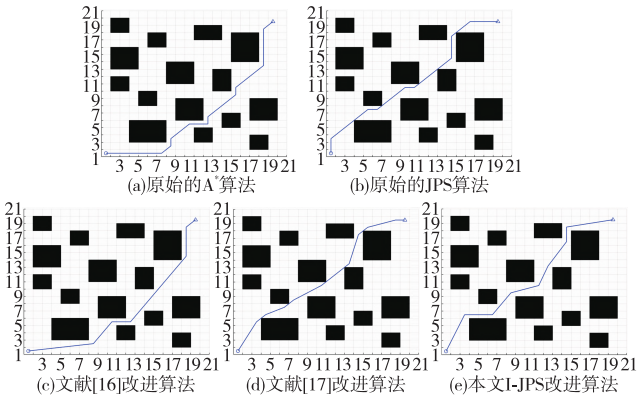


图 8　单机器人实验仿真

Fig. 8　Single robot experimental simulation

由实验仿真结果可以看出,仿真通过对比不同路径规划算法在任务性能上的表现,发现本文的 I-JPS 算法相较于其他算法具有显著优势。在计算规划耗时方面,I-JPS 算法仅需耗时原始 A* 算法的 11.96%、文献[16]算法的 17.91%、JPS 算法的 33.88%。路径长度上,I-JPS 算法相比其他算法仍有一定优势,仅为原始 A 算法的 89.94%、文献[16]算法的 93.77%。在拐点数和搜索节点数方面,I-JPS 算法相较于其他算法分别仅有原始 A* 算法的 70% 的拐点数和 62.6% 的搜索节点数,文献[17]的 70% 的拐点数。相比于文献[17]存在大量危险节点,靠近障碍物的情况,I-JPS 算法的危险节点个数为 0,极大地提高了路径规划的安全性和稳定性。以上结果表明,在单机器人路径规划时,I-JPS 算法在综合性能上表现卓越,是一种高效的路径规划算法。

表 1　算法对比

Tab. 1　Algorithm comparison

算法	时间/ms	路径长度/m	拐点数	搜索节点
原始的 A* 算法	80.634	30.142 1	10	219
原始的 JPS 算法	26.568	27.384 3	8	26
文献[16]算法	56.392	28.907 4	5	89
文献[17]算法	20.195	27.084 4	10	125
I-JPS 算法	9.649	27.093 6	7	26

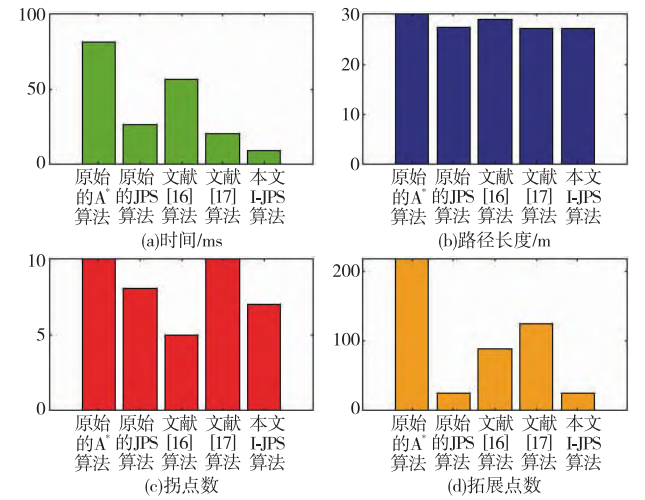


图 9　柱状图 1

Fig. 9　Bar chart 1

下面进行单机器人局部路径规划仿真,表 2 为移动机器人运动学参数的设置,评价函数的五个参数分别取值 $a_1=0.3$、$a_2=0.5$、$a_3=0.2$、$a_4=0.5$、$a_5=0.5$。

表 2　运动学参数设置

Tab. 2　Kinematic parameter settings

运动学参数	值	运动学参数	值
最高速度	2 m/s	旋转加速度	50 rad/s^2
最高旋转速度	20 rad/s	速度分辨率	0.01 m/s
加速度	0.2 m/s^2	转速分辨率	1 rad/s

如图 10 所示,在动态环境中对 I-JPS 算法与 DWA 耦合的移动机器人路径规划仿真实验,分别设置了一个移动障碍物和三个未知障碍物,黄色方块为移动障碍物,坐标为(13,14),红色线段代表移动障碍物的运动轨迹,灰色方块代表移动障碍物,其坐标分别为(2,7)(8,6)和(9,3)。机器人通过躲避已知障碍物、未知障碍物和动态障碍物最后顺利到达终点,其最终运行的轨迹如图 11(a)所示。通过本文改进的 DWA,其运行的姿态角度变化如图 11(b)所示,线速度和角速度的变化如图 11(c)所示。

4.2　多机器人路径规划仿真实验

多机器人可以同时执行任务,相对于单机器人,能更迅速

中国知网　https://www.cnki.net

地完成一系列任务，这对于需要在有限时间内完成的任务非常重要。下面进行多机器人的路径规划实验仿真，首先在 30 × 30 的复杂环境下进行静态障碍物的实验仿真。

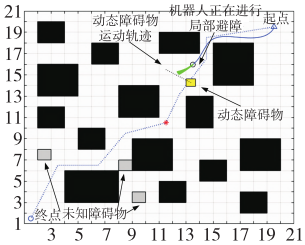
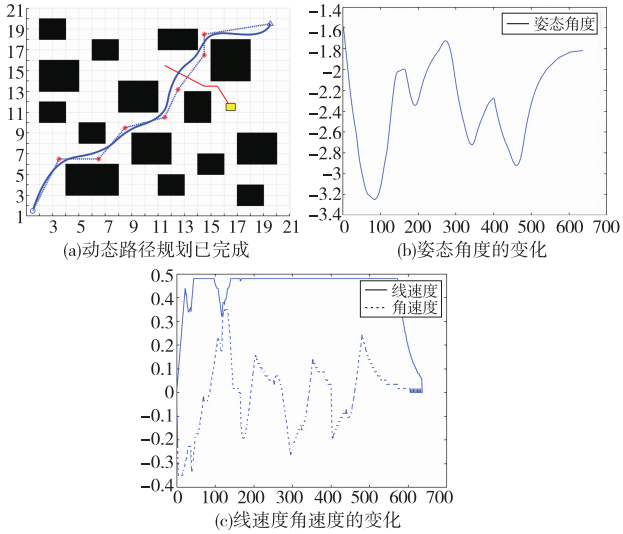


图 10　局部路径规划

Fig. 10　Local path planning



(a)动态路径规划已完成　(b)姿态角度的变化

(c)线速度角速度的变化

图 11　参数变化

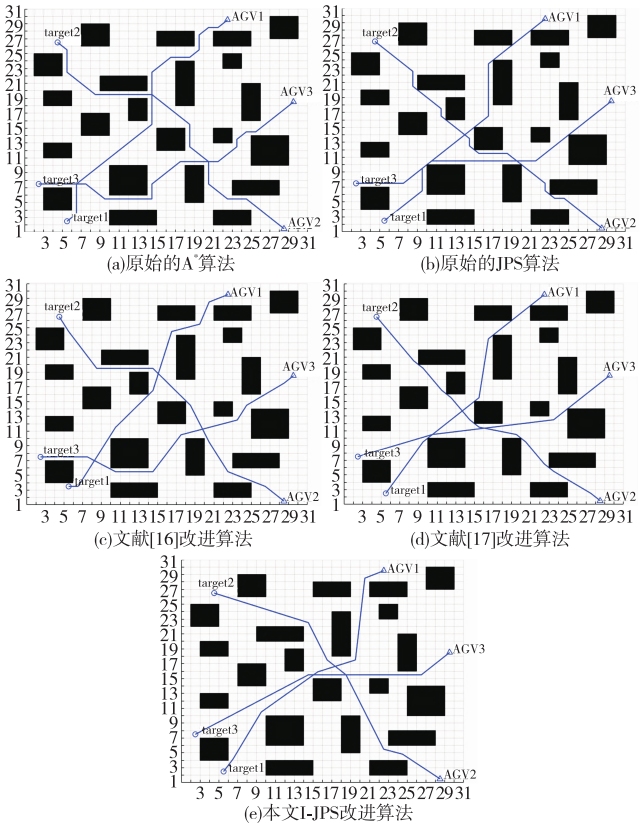
Fig. 11　Parameter changes

如图 12 所示，通过对比两种原始算法、两种改进算法和本文改进后的 I-JPS 算法可以看出，原始的 A* 算法的拐点、冗余节点过多。原始 JPS 算法的危险节点过多、拐点相对偏多。文献[16]算法提高了路径规划的安全性，危险节点大大减少，但规划出的路径仍有很大提升的空间。文献[17]算法虽然规划出的路径相当短，但危险节点并未作出及时的处理。由图 12 可知，本文提出的 I-JPS 算法在考虑去除危险节点的情况下，保证规划出的路径最优、用时最短且拐点最少，达到了理想的效果。

如图 13 和表 3 所示，表 3 中的时间、路径长度、拓展点数和总的拓展节点均为三条移动机器人路线所得数据的总和，实验结果均为进行 30 次实验所取得的平均值。根据实验数据可以得出，在多机器人处于静态障碍物的复杂环境下，本文改进后 I-JPS 算法在时间、拐点数、路径长短和拓展点数均优于原始的算法和文献[16,17]的算法。

下面进行多机器人动态路径规划的研究。如图 14 所示，三角形代表机器人的起点，星号代表机器人的终点，图 14(a)中，三个机器人分别开始了它们的动态路径规划，机器人 1 即将开始躲避黄色的移动动态障碍物，机器人 2 没有碰到动态障碍物，无须躲避，机器人 3 已经完成了动态障碍物的躲避。其中，红色方块代表未知障碍物。如图 14(b)所示，多机器人按照避障优先级，完成了三个机器人在中心点相遇的过程。

如图 15 所示，其中(a)为多机器人已完成路径规划，动态路径的长度分别为 34.172 0、29.728 0、36.478 0。姿态角度分别为 −2.132 7、−2.741 4、2.335 8。图(b)为三个机器人线速度的变化，图(c)为三个机器人角速度的变化，图(d)为三个机器人角度的变化。综上所述，本文提出的 I-JPS 算法可以为机器人在复杂环境中快速且有效地规划出全局路径。同时，当路径上出现动态障碍物以及未知障碍物时，机器人也能够通过局部路径规划顺利绕开障碍物，确保自身安全。最后，对于多机器人系统之间的协调问题，算法也能够根据动态优先级规则完成机器人之间的避让。实验结果表明，该改进算法可以很好地适应于各种简单以及复杂的环境，为移动机器人规划出一条安全无碰撞路径，且在没有人为干涉的前提下处理各种突发状况。



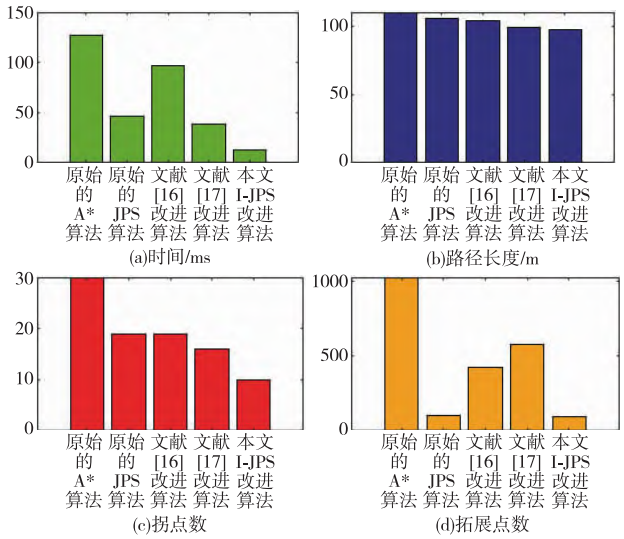
(a)原始的A*算法　(b)原始的JPS算法

(c)文献[16]改进算法　(d)文献[17]改进算法

(e)本文I-JPS改进算法

图 12　多机器人实验仿真

Fig. 12　Multi-robot experimental simulation



(a)时间/ms　(b)路径长度/m

(c)拐点数　(d)拓展点数

图 13　柱状图 2

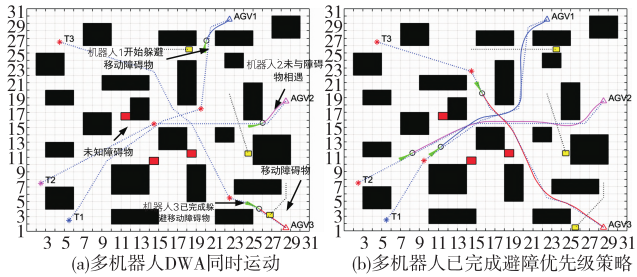
Fig. 13　Bar chart 2

表 3　算法对比图 2

Tab. 3　Algorithm comparison chart2

算法	总时间/ms	路径总长度/m	总拐点数	搜索节点总数
原始的 A* 算法	126.594	110.397 0	30	1025
原始的 JPS 算法	46.316	105.710 6	19	106
文献[16]算法	96.357	104.182 4	19	426
文献[17]算法	38.143	98.913 5	16	581
I-JPS 算法	13.259	97.870 1	10	97

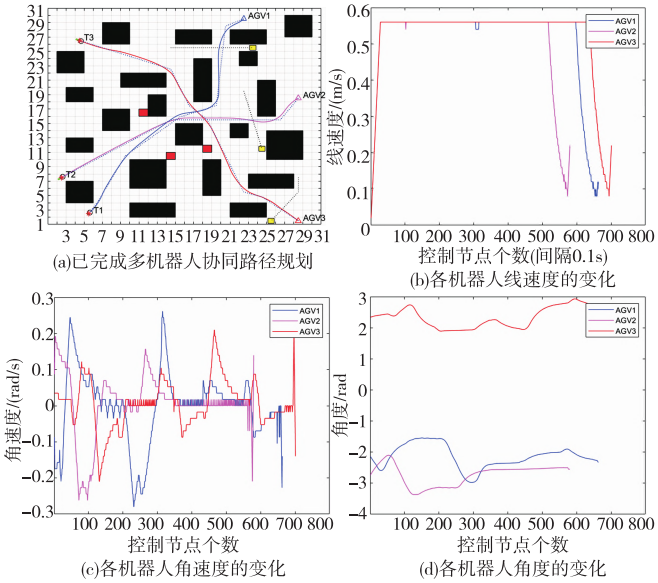
中国知网　https://www.cnki.net



(a)多机器人DWA同时运动　(b)多机器人已完成避障优先级策略

图 14　局部规划过程

Fig. 14　Local planning process



(a)已完成多机器人协同路径规划　(b)各机器人线速度的变化

(c)各机器人角速度的变化　(d)各机器人角度的变化

图 15　参数变化

Fig. 15　Parameter changes

5　结束语

本文针对传统的 JPS 算法在路径规划中无效跳点过多、路径不安全等缺点,提出了一种改进的跳点搜索算法,称为 I-JPS 算法。新算法通过引入向量叉积公式对多余节点进行去除,同时通过改进代价函数进一步提高路径规划的时间,并增大了机器人与障碍物之间的安全距离。对于未知和动态障碍物,本文引入了 DWA 并进行进一步的改进,改进后的 DWA 可以让多机器人在路径规划时避免发生碰撞并遵循避障有优先级原则。实验结果表明,改进后的多种算法相较于改进前的,得到了极大的提升。同时本文的不足之处是多机器人运行时计算量较大,DWA 动态局部路径规划时,机器人运行速度比较慢,偶尔会出现局部路径过长等现象。

参考文献:

[1] 陈浩, 陈珺, 刘飞. 基于自主探索的移动机器人路径规划研究[J]. 计算机工程. [2024-05-17]. https://www.cnki.com.cn/Article/CJFDTotalJSJC20240419005.htm. (Chen Hao, Chen Jun, Liu Fei. Research on mobile robot path planning based on autonomous exploration [J/OL]. Computer Engineering. [2024-05-17]. https://www.cnki.com.cn/Article/CJFDTotalJSJC20240419005.htm.)

[2] 贺志刚, 毛剑琳, 杨邹,等. 一种面向地下管网环境的多机器人路径规划算法[J]. 机器人, 2024, 46(1): 94-104, 117. (He Zhigang, Mao Jianling, Yang Zou, *et al.* A Multi robot path planning algorithm for underground pipeline network environment [J]. Robot, 2024, 46(1): 94-104, 117.)

[3] Dorigo M, Socha K. An introduction to ant colony optimization [M]// Handbook of Approximation Algorithms and Metaheuristics. 2018: 395-408.

[4] Duchoň F, Babinec A, Kajan M, *et al.* Path planning with modified A star algorithm for a mobile robot [J]. Procedia Engineering, 2014, 96: 59-69.

[5] Jain M, Saihjpal V, Singh N, *et al.* An overview of variants and advancements of PSO algorithm [J]. Applied Sciences, 2022, 12(17): 8392.

[6] Dhulkefl E, Durdu A, Terzioğlu H. Dijkstra algorithm using UAV path planning [J]. Konya Journal of Engineering Sciences, 2020, 8: 92-105.

[7] Reddy G T, Reddy M P K, Lakshmanna K, *et al.* Hybrid genetic algorithm and a fuzzy logic classifier for heart disease diagnosis [J]. Evolutionary Intelligence, 2020, 13: 185-196.

[8] Liu Lisang, Lin Jiafeng, Shi Peng, *et al.* Path planning for smart car based on Dijkstra algorithm and dynamic window approach [J]. Wireless Communications and Mobile Computing, 2021, 2021: 1-12.

[9] Chen Yanli, Bai Guiqiang, Zhan Yin, *et al.* Path planning and obstacle avoiding of the USV based on improved ACO-APF hybrid algorithm with adaptive early-warning [J]. IEEE Access, 2021, 9: 40728-40742.

[10] 朱敏, 胡若海, 卞京. 基于改进蚁群算法的移动机器人路径规划[J]. 现代制造工程, 2024(3): 38-44. (Zhu Min, Hu Ruohai, Bian Jing, *et al.* Mobile robot path planning based on improved ant colony algorithm [J]. Modern Manufacturing Engineering, 2024(3): 38-44.)

[11] 周滔, 赵津, 胡秋霞,等. 复杂环境下移动机器人全局路径规划与跟踪[J]. 计算机工程, 2018, 44(12): 208-214. (Zhou Tao, Zhao Jin, Hu Qiuxia, *et al.* Global path planning and tracking of mobile robots in complex environments [J]. Computer Engineering, 2018, 44(12): 208-214.)

[12] 刘荣华, 王欣, 吴迪,等. 改进双向动态 JPS 算法的移动机器人全局路径规划[J]. 计算机应用研究, 2024, 41(4): 1117-1122. (Liu Ronghua, Wang Xing, Wu Di, *et al.* Global path planning for mobile robots using improved bidirectional dynamic JPS algorithm [J]. Application Research of Computers, 2024, 41(4): 1117-1122.)

[13] 苗红霞, 郭章旺, 齐本胜,等. 基于并行-交替式双向 JPS 算法的机器人路径规划[J]. 计算机测量与控制, 2022, 30(7): 233-239. (Miao Hongxia, Guo Zhangwang, Qi Bensheng, *et al.* Robot path planning based on parallel alternating bidirectional JPS algorithm [J]. Computer Measurement & Control, 2022, 30(7): 233-239.)

[14] 林信川. 跳点搜索融合双向并行蚁群算法的 AGV 路径规划研究[J]. 南京信息工程大学学报, 2024, 16(4): 504-512. (Lin Xinchuan. Research on AGV path planning with jump point search fusion and bidirectional parallel ant colony algorithm [J]. Journal of Nanjing University of Information Technology, 2024, 16(4): 504-512.)

[15] 孙岩霆, 王荣杰, 蒋德松. 融合 A* 与 DWA 算法的水面船艇动态路径规划[J]. 仪器仪表学报, 2024, 45(1): 301-310. (Sun Yanting, Wang Rongjie, Jiang Desong. Dynamic path planning of surface craft by integrating A * and DWA algorithms [J]. Chinese Journal of Scientific Instrument, 2024, 45(1): 301-310.)

[16] 黄智榜, 胡立坤, 张宇,等. 基于改进跳点搜索策略的安全路径研究[J]. 计算机工程与应用, 2021, 57(1): 56-61. (Huang Zhibang, Hu Likong, Zhang Yu, *et al.* Research on secure path based on improved skip point search strategy [J]. Computer Engineering and Applications, 2021, 57(1): 56-61.)

[17] 龚轶群, 朱琳. 医院场景下多机器人系统双层路径规划算法[J/OL]. 系统仿真学报. [2024-03-19]. (Gong Yiqun, Zhu Lin. A double layer path planning algorithm for multi robot systems in hospital scenarios [J/OL]. Journal of System Simulation. [2024-03-19].)

中国知网　https://www.cnki.net